

Lesson 4 — Snake

One lesson: 2 hours 30 minutes · keep this sheet

Name _____

Class _____

Date _____

Cheat sheet

A snake is a list

```
snake = [(5, 5), (4, 5), (3, 5)]
#         head   body   tail
```

Moving: add a head, drop a tail

```
snake.insert(0, new_head) # grow at the
FRONT
snake.pop()               # drop the BACK
```

No other segment is touched.

Growing: just skip the pop

```
if new_head == food:
    score += 10
    food = place_food()
    # NO pop() - that is the whole growth
    mechanic
else:
    snake.pop()
```

Directions are pairs

```
RIGHT = (1, 0); LEFT = (-1, 0)
DOWN  = (0, 1); UP   = (0, -1)

dc, dr = direction
new_head = (head[0] + dc, head[1] + dr)
```

No `if` statements about direction, anywhere.

No reversing

```
if (new[0] == -direction[0] and
    new[1] == -direction[1]):
    return # reject it
next_direction = new
```

Store the **request**; apply it when the snake **moves**. Otherwise two fast presses let the snake reverse into itself.

Walls and self

```
if not (0 <= x < COLUMNS and 0 <= y < ROWS):
    state = "GAME_OVER"

if new_head in snake: # BEFORE insert!
    state = "GAME_OVER"
```

Food on a free square

```
while True:
    spot = (randint(0, COLUMNS-1),
            randint(0, ROWS-1))
    if spot not in snake:
        return spot
```

Tick slower than you draw

```
move_timer += dt
if move_timer >= MOVE_DELAY:
    move_timer -= MOVE_DELAY # not = 0
    move_snake()
```

Render at 60 fps; step the snake a few times a second.

Why a grid has no collision bugs

Two squares are either **equal or not**. No overlap test, no rounding, no tunnelling.

Section A — Recall

A1. Describe, in words, what the two lines that move the snake do. Why is no other segment touched? [3 marks]

A2. How does the snake grow? Give the one-line answer. [2 marks]

A3. Why does the game store direction as `(1, 0)` rather than the string `"right"`? [3 marks]

A4. Why does Snake need no overlap test, when the web track's Pong needed four conditions? [3 marks]

A5. Why `move_timer -= MOVE_DELAY` rather than `move_timer = 0`? [2 marks]

Section B — Predict the output

B1. The snake is `[(5,5), (4,5), (3,5)]` and the direction is `DOWN`. Write the list after one normal move, and then after one move where it eats. [4 marks]

B2. What happens, and why? Be precise about which square causes it. [4 marks]

```
snake.insert(0, new_head)
if new_head in snake:
    state = "GAME_OVER"
```

B3. `MOVE_DELAY` is 0.12 and the game loop runs at 60 fps. Roughly how many frames pass between snake steps? How many squares does the snake move per second? [3 marks]

B4. The snake is moving **right**. A player presses **Up** and then **Left** very quickly, both between two ticks. Describe what happens with each version, and which is correct. [4 marks]

```
# version A
def set_direction(new):
    global direction
    direction = new

# version B
def set_direction(new):
    global next_direction
    next_direction = new
```

Section C — Find and fix the bug

C1. The snake grows by one square on every single move, whether it eats or not. What is wrong? [3 marks]

C2. Food sometimes appears and seems to vanish instantly, or cannot be eaten at all. What is missing from `place_food`, and why does it produce *that* symptom? [4 marks]

C3. This raises `TypeError: 'tuple' object does not support item assignment`. Why, and what should it be? [3 marks]

```
head = snake[0]
head[0] = head[0] + 1
```

Section D — Build it

- ☐ **Checkpoint 1.** Draw a grid of squares from a list of `(column, row)` positions. Write `grid_to_pixels` and check that square `(0, 0)` is in the top-left corner.
- ☐ **Checkpoint 2.** Make the snake move: `insert` a new head, `pop` the tail, on a timer. It should glide across the screen at a readable speed.
- ☐ **Checkpoint 3.** Add the arrow keys using direction pairs, and reject reversals. Use `next_direction`, not `direction`.
- ☐ **Checkpoint 4.** Add food, eating and growth. Make sure food never spawns on the snake.
- ☐ **Checkpoint 5.** Add wall and self-collision, a score, and a game-over screen with a restart.

- ☐ **Checkpoint 6.** Make the head a different colour from the body, and make the snake speed up slightly as it grows.

ORDER MATTERS IN THIS LESSON

Check self-collision **before** `insert`, not after. Afterwards, the new head is in the list, so it finds itself and the game ends on the first move.

Section E — Design challenge

NO RIGHT ANSWER

E1. `place_food` guesses until it finds an empty square. If the snake filled 99% of the board, roughly how many guesses would it need? Design a method that **never** guesses, and say what that one costs.

E2. `new_head in snake` checks every segment, so a 200-long snake does 200 comparisons per move. At what length would you start to care? What could you store **instead** to make the check instant? (You know a Python type that does this.)

E3. Snake traditionally has no win condition. What *should* happen when the snake fills the whole board? Can your code even detect it?

E4. Design a difficulty curve. There are at least **four** different things you could change as the game goes on. Find them, then say which you think feels best and why.

Stretch goals

1. Speed up as the snake grows. Find a curve that stays fun rather than becoming impossible.
2. Wrap-around walls using `%`.
3. A high score saved to a file.
4. Obstacles inside the playing area. How much code had to change?

5. **Two players.** What happens if both heads enter the same square on the same tick? You will have to decide something the rules do not cover.
6. Food that moves.